

TALLER 3 – SEMANA 3
IMPLEMENTACIÓN PATRON SINGLETON Y FACTORY

OSCAR YAIR PARDO PINEDA
JORGE ALFREDO LEAL CRUZ

PRESENTADO A:
ELIECER ONTERO OJEDA

GRUPO: E196

INGENIERIA DE SISTEMAS



INSTITUCION UNIVERSITARIA TECNOLOGICA DE SANTANDER
BUCARAMANGA

2026 – 1

PROYECTO A EJECUTARSE

PROYECTO SECTOR LOGÍSTICO/TRANSPORTE

Sistema de Gestión de Flotas

▼ Autores

Estudiante 1: Oscar Yair Pardo Pineda

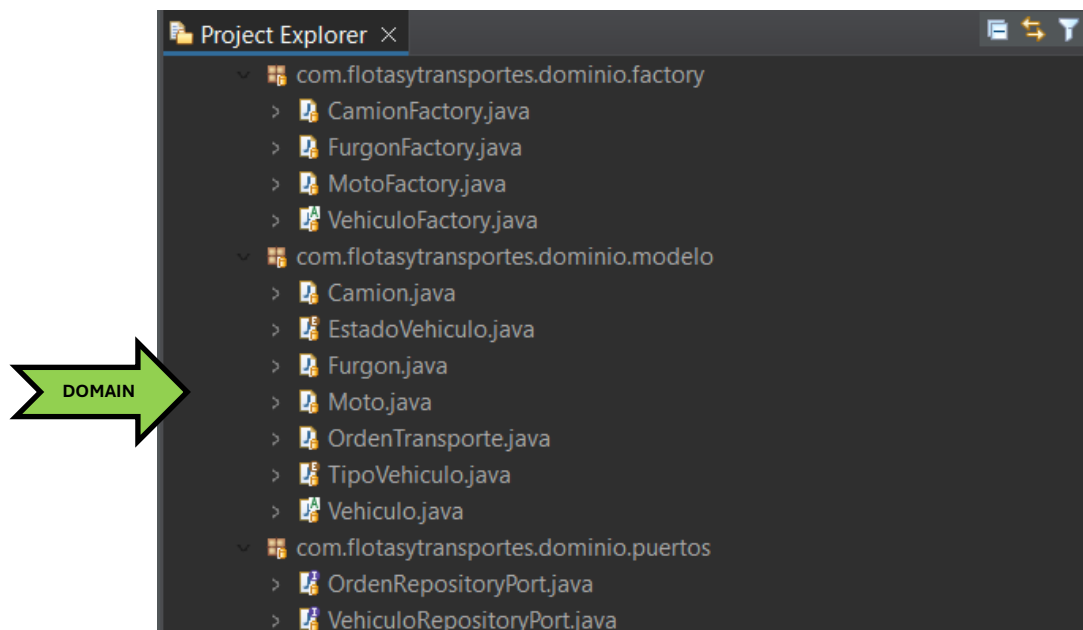
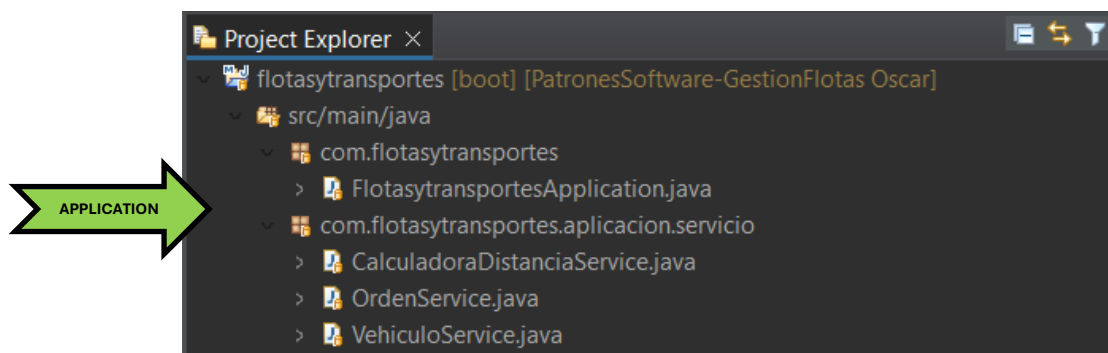
Estudiante 2: Jorge Alfredo Leal Cruz

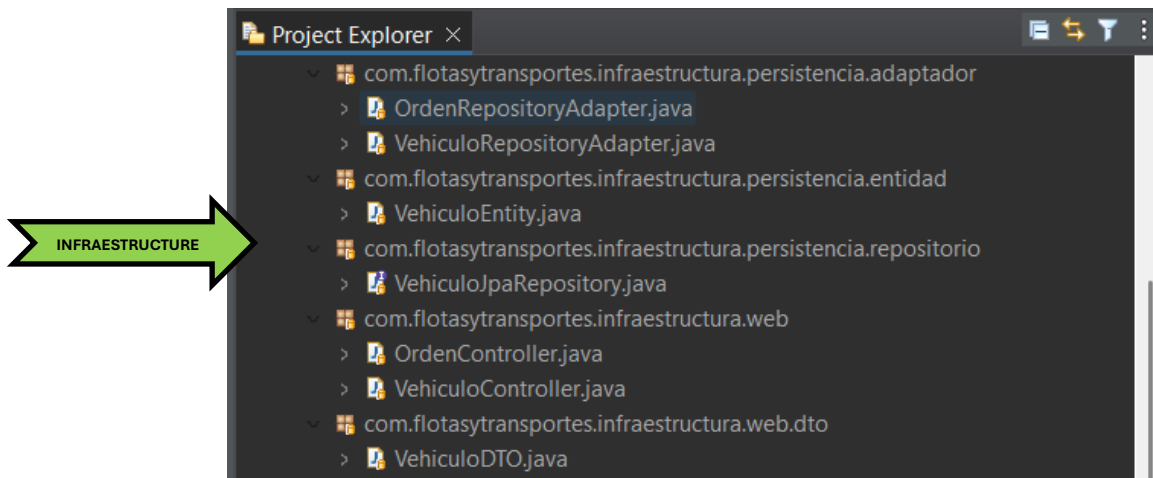
Docente: Eliecer Montero Ojeda

Unidades Tecnológicas de Santander - 2026

SpringBoot 3.x Java 17 MySQL Database Status En Desarrollo Proyecto Académico UTS Logística SOLID Principios
Sistema Gestión de Flotas

ESTRUCTURA DEL PROYECTO





ARQUITECTURA HEXAGONAL

La Arquitectura Hexagonal, también conocida como Arquitectura de Puertos y Adaptadores, es un modelo arquitectónico propuesto por Alistair Cockburn cuyo objetivo principal es aislar completamente la lógica de negocio de cualquier dependencia tecnológica externa.

Su principio fundamental establece que:

- El núcleo del sistema (dominio) no debe depender de frameworks, bases de datos ni interfaces externas.
- Las dependencias deben apuntar siempre hacia el interior.

En este modelo, el dominio se ubica en el centro de la arquitectura y representa la parte más estable y crítica del sistema: las reglas del negocio. A diferencia de las arquitecturas tradicionales en capas donde el dominio suele terminar acoplado a tecnologías como frameworks web o librerías de persistencia la arquitectura hexagonal protege la lógica del negocio colocándola en el núcleo y obligando a que todo lo externo dependa de él, y no al revés.

El sistema se organiza alrededor de un núcleo (dominio) y se conecta con el mundo exterior mediante puertos y adaptadores.

- **Puertos:** Son interfaces definidas por el dominio que establecen contratos de comunicación.
 - **Puertos de entrada:** representan los casos de uso que el sistema expone.
 - **Puertos de salida:** representan servicios externos que el dominio necesita (persistencia, servicios externos, etc.).

- **Adaptadores:** Son implementaciones concretas de esos puertos. Permiten que tecnologías específicas (como controladores REST, bases de datos o APIs externas) interactúen con el sistema sin modificar el dominio.

Este enfoque permite que el sistema tenga múltiples puntos de entrada y salida (por ejemplo, API REST, consola, mensajería, base de datos), todos conectados al núcleo mediante contratos bien definidos, sin que el dominio conozca detalles técnicos.

CAPA DE DOMINIO

La capa de dominio constituye el centro arquitectónico del sistema. Está conformada por los siguientes paquetes:

- `com.flotasytransportes.dominio.modelo`
- `com.flotasytransportes.dominio.factory`
- `com.flotasytransportes.dominio.puertos`

Aquí se concentra la lógica real del negocio logístico.

MODELO

En el paquete `dominio.modelo` se encuentran las entidades principales del sistema, tales como:

- `Vehiculo`
- `Camion`
- `Furgon`
- `Moto`
- `OrdenTransporte`
- `EstadoVehiculo`
- `TipoVehiculo`

Estas clases representan el modelo conceptual del negocio de flotas. No son simples estructuras de datos, sino que encapsulan comportamiento y reglas propias del dominio, como los estados operativos de un vehículo, la relación entre órdenes y unidades de transporte, o la clasificación por tipo.

Un aspecto fundamental es que estas clases no contienen anotaciones de Spring ni de JPA. Esto demuestra que el dominio se mantiene completamente aislado de la tecnología, cumpliendo el principio central de la arquitectura hexagonal: la lógica de negocio no depende del framework.

FACTORY

En el paquete `dominio.factory` se encuentran clases como:

- VehiculoFactory
- CamionFactory
- FurgonFactory
- MotoFactory

Estas clases encapsulan la lógica de creación de los distintos tipos de vehículos. Desde el punto de vista arquitectónico, esta decisión es relevante porque la creación de entidades forma parte del negocio y no debe delegarse a la infraestructura.

Al centralizar la construcción de objetos dentro del dominio:

- Se evita la dispersión de lógica de instanciación.
- Se mantiene bajo acoplamiento.
- Se facilita la extensión del sistema ante nuevos tipos de vehículos.

PORT

El paquete dominio.puertos contiene interfaces como:

- OrdenRepositoryPort
- VehiculoRepositoryPort

Estos elementos representan los puertos de salida del sistema. Es decir, son contratos definidos por el dominio para interactuar con el exterior, particularmente con la persistencia.

El dominio declara qué necesita (guardar vehículos, consultar órdenes, persistir información), pero no define cómo se implementa. La implementación concreta se delega a la infraestructura.

Este diseño materializa el principio de Inversión de Dependencias, ya que la infraestructura depende de interfaces del dominio, y no al contrario.

CAPA DE APLICACIÓN

La capa de aplicación está representada por el paquete:

- com.flotasytransportes.aplicacion.servicio

Incluye clases como:

- VehiculoService
- OrdenService
- CalculadoraDistanciaService

Esta capa no contiene lógica técnica ni detalles de persistencia. Su función es ejecutar los casos de uso del sistema.

Por ejemplo, un servicio puede:

1. Recibir una solicitud proveniente de la capa web.
2. Utilizar una fábrica del dominio para crear una entidad.
3. Invocar un puerto de repositorio para persistir la información.
4. Coordinar el flujo completo del caso de uso.

Es importante destacar que la capa de aplicación no es infraestructura. No maneja directamente la base de datos ni detalles de HTTP. Su responsabilidad es coordinar el dominio, manteniendo la coherencia del flujo operativo.

CAPA DE INFRAESTRUCTURA

La capa de infraestructura contiene las implementaciones técnicas que permiten conectar el dominio con el entorno externo.

ADAPTADORES

En el paquete `infrastructure.persistencia.adaptador` se encuentran:

- `OrdenRepositoryAdapter`
- `VehiculoRepositoryAdapter`

Estas clases implementan las interfaces definidas en `dominio.puertos`. Aquí se concreta la comunicación con la base de datos.

ENTIDADES

El paquete `infrastructure.persistencia.entidad` contiene clases como:

- `VehiculoEntity`

Es importante señalar que estas entidades son distintas a las entidades del dominio. Esto evita que el modelo del negocio quede contaminado con anotaciones técnicas, manteniendo una separación limpia entre modelo conceptual y modelo de base de datos.

REPOSITORIOS

En `infrastructure.persistencia.repositorio` se encuentra:

- `VehiculoJpaRepository`

Aquí aparece el uso de tecnologías como Spring Data JPA y Spring Boot.

Estas dependencias están correctamente ubicadas en la capa externa, cumpliendo el principio de independencia tecnológica del dominio.

PATRÓN SINGLETON

El Singleton es un patrón de diseño creacional cuyo objetivo es garantizar que una clase tenga una única instancia durante todo el ciclo de vida de la aplicación y proporcionar un punto de acceso global controlado a dicha instancia.

El patrón se utiliza cuando:

- No tiene sentido crear múltiples instancias del mismo objeto.
- El componente coordina lógica central del sistema.
- Se desea controlar el acceso a recursos compartidos.

Tradicionalmente, en Java, se implementa mediante:

- Constructor privado.
- Atributo estático.
- Método público getInstance().

Sin embargo, en aplicaciones modernas con contenedores IoC, como **SPRING**, el patrón puede implementarse de forma gestionada por el framework.

En este proyecto, el patrón Singleton no fue implementado manualmente, sino que se utiliza el Singleton gestionado por el contenedor de Spring.

En Spring Framework, el alcance por defecto de los componentes es patrón **SINGLETON**.

Esto significa que:

- Se crea una única instancia.
- Se reutiliza en toda la aplicación.
- No se permite la creación manual repetida.



IMPORTANTE

El patrón Singleton se evidencia en las clases anotadas con la siguiente sentencia:

- @Service
- @Repository
- @Component

```

CalculadoraDistanciaService.java ×
1 package com.flotasytransportes.aplicacion.servicio;
2
3 import org.springframework.stereotype.Service;
4
5 @Service
6 public class CalculadoraDistanciaService {
7
8     private static final double RADIO_TIERRA_KM = 6371;
9
10    public double calcularDistancia(double lat1, double lon1, double lat2, double lon2) {
11
12        double dLat = Math.toRadians(lat2 - lat1);
13        double dLon = Math.toRadians(lon2 - lon1);
14
15        double a = Math.sin(dLat / 2) * Math.sin(dLat / 2) + Math.cos(Math.toRadians(lat1))
16            * Math.cos(Math.toRadians(lat2)) * Math.sin(dLon / 2) * Math.sin(dLon / 2);
17
18        double c = 2 * Math.asin(Math.sqrt(a));
19
20        return RADIO_TIERRA_KM * c;
21    }
22 }
23

```

```

OrdenRepositoryAdapter.java ×
1 package com.flotasytransportes.infraestructura.persistencia.adaptador;
2
3 import org.springframework.stereotype.Repository;
4
5 @Repository
6 public class OrdenRepositoryAdapter implements OrdenRepositoryPort {
7
8     @Override
9     public OrdenTransporte guardar(OrdenTransporte orden) {
10
11        // lógica de guardado
12        return orden;
13    }
14 }
15

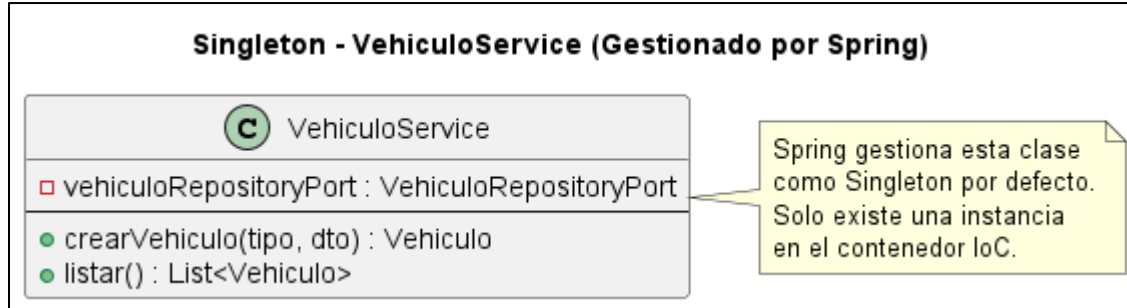
```

El patrón Singleton se utiliza en tu Sistema de Gestión de Flotas porque:

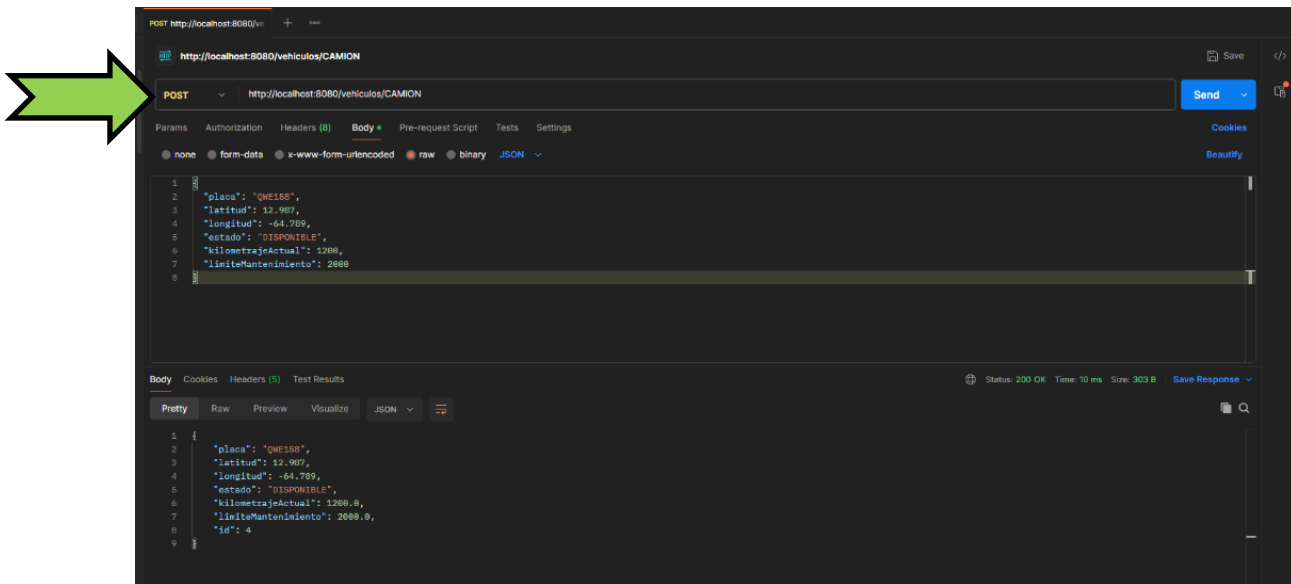
- Los servicios representan coordinadores centrales del negocio.
- Los repositorios gestionan recursos compartidos.
- No se requiere estado independiente por instancia.
- Se busca eficiencia y coherencia global.
- El framework utilizado lo implementa de manera natural y segura.

En este contexto, el Singleton no es una elección arbitraria, sino una consecuencia lógica del diseño del sistema y de la naturaleza de sus componentes.

UML



PRUEBA



La imagen anterior muestra la ejecución de una petición HTTP al endpoint expuesto por VehiculoController, el cual utiliza internamente la clase VehiculoService, anotada con `@Service`.

Durante la ejecución de múltiples solicitudes desde Postman, se imprimió en consola el hashCode de la instancia de VehiculoService. Se evidenció que el identificador de la instancia permanece constante en cada petición, lo que confirma que el contenedor de Spring crea una única instancia de la clase y la reutiliza durante todo el ciclo de vida de la aplicación.

Este comportamiento demuestra la aplicación del patrón Singleton en la capa de servicios, ya que existe una sola instancia compartida que coordina la lógica de negocio relacionada con la gestión de vehículos.

PATRÓN FACTORY METHOD

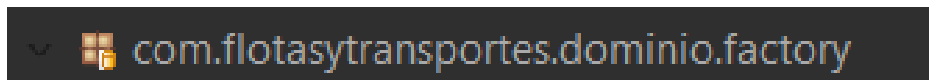
El Factory Method es un patrón de diseño creacional cuyo propósito es definir una interfaz para la creación de objetos, permitiendo que las subclases decidan qué clase concreta instanciar.

En lugar de crear objetos directamente mediante *new ClaseConcreta()* la creación se delega a una fábrica que encapsula la lógica de instanciación.

Este patrón se utiliza cuando:

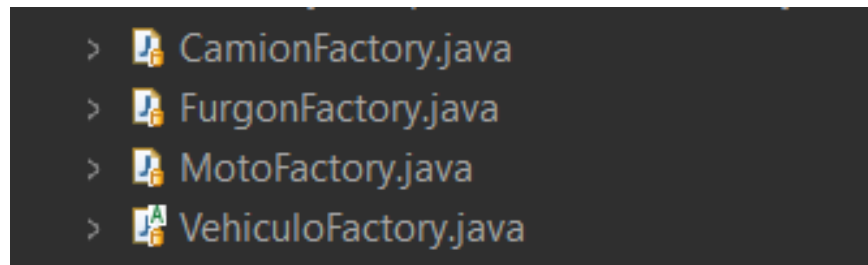
- Existen múltiples variantes de un mismo tipo de objeto.
- La creación depende de una condición.
- Se desea reducir el acoplamiento entre el cliente y las clases concretas.

En este proyecto, el patrón Factory Method se encuentra en el paquete:



```
com.flotasytransportes.dominio.factory
```

Clases involucradas:



```
> CamionFactory.java  
> FurgonFactory.java  
> MotoFactory.java  
> VehiculoFactory.java
```

Estas clases encapsulan la lógica de creación de los distintos tipos de vehículos.

En el sistema existen distintos tipos de vehículos:

- Camión
- Furgón
- Moto

Cada uno puede tener características o comportamientos particulares.

Si la creación se hiciera directamente en el servicio con múltiples condicionales:

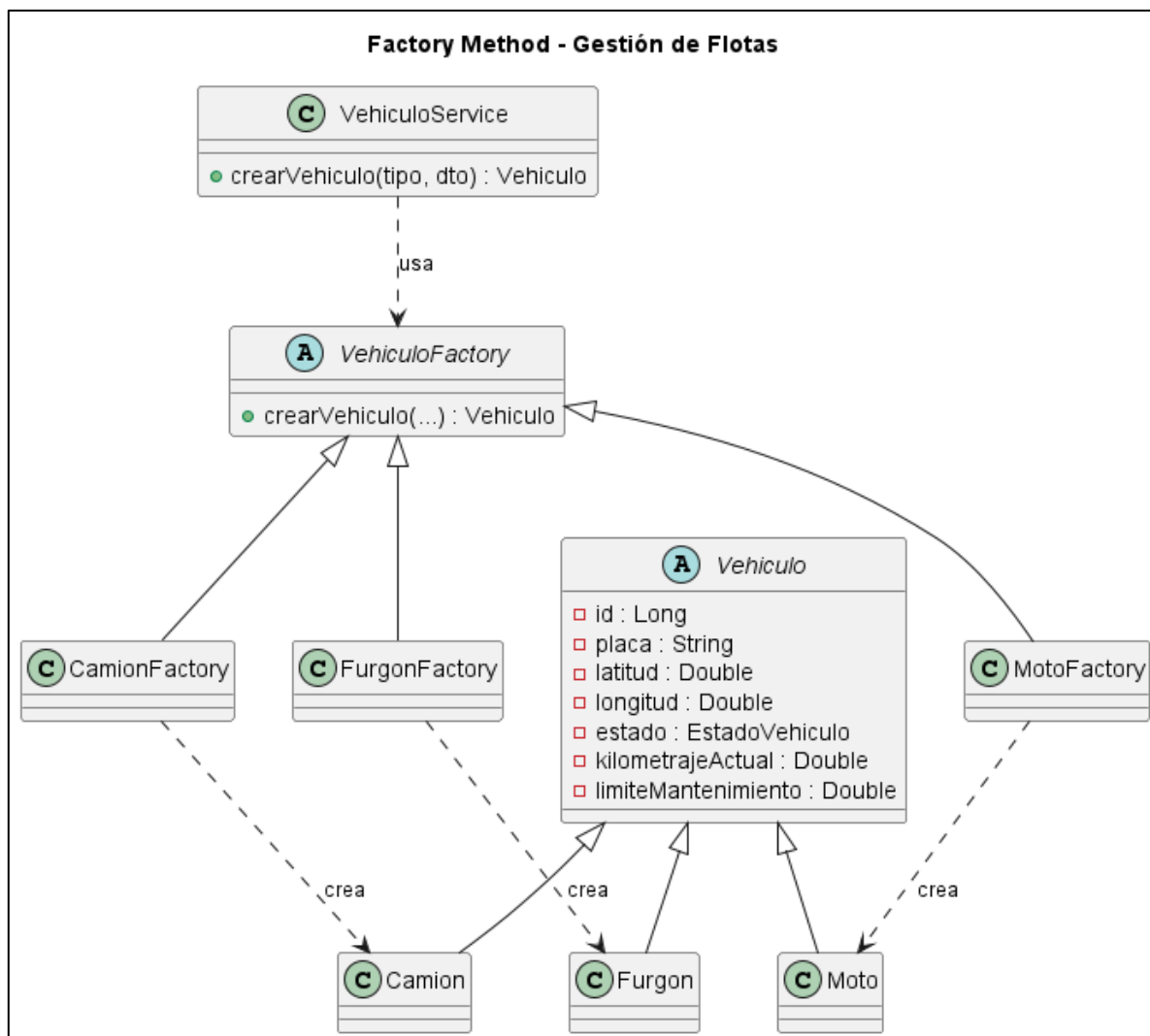
```
if(tipo.equals("CAMION")) {  
    return new Camion(...);  
} else if(tipo.equals("FURGON")) {  
    return new Furgon(...);  
}
```

Esto generaría:

- Alto acoplamiento
- Violación del principio Open/Closed
- Código difícil de mantener
- Lógica de creación dispersa

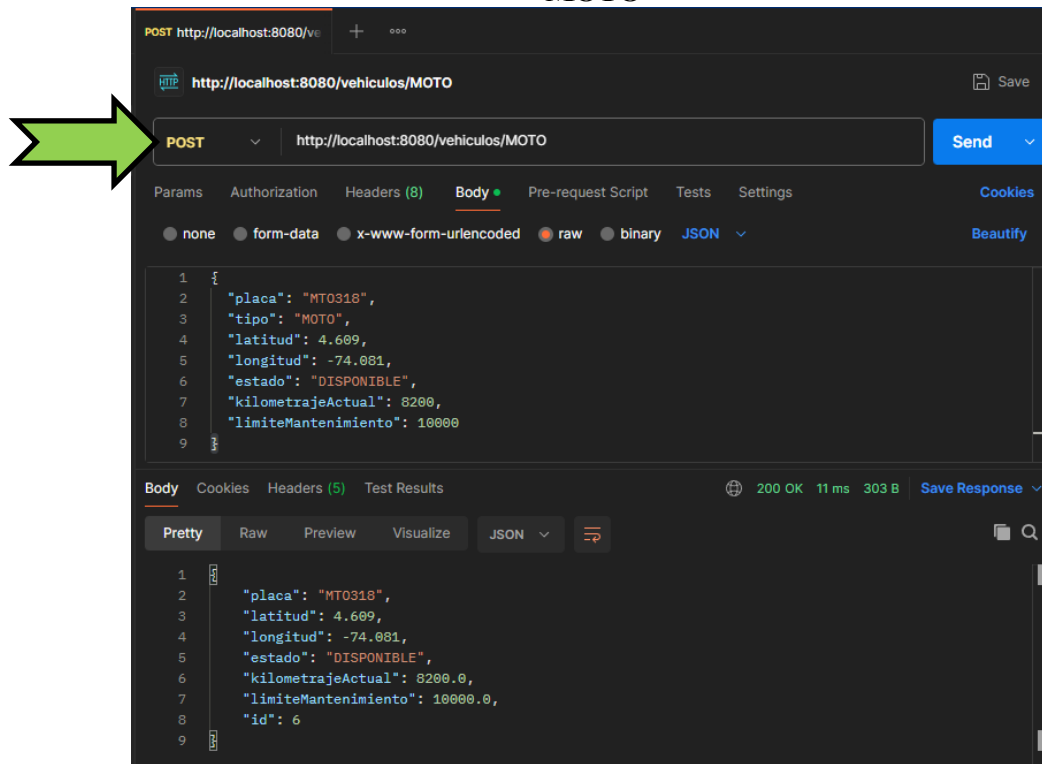
El Factory Method centraliza esa responsabilidad.

UML



PRUEBA

MOTO



POST http://localhost:8080/vehiculos/MOTO

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

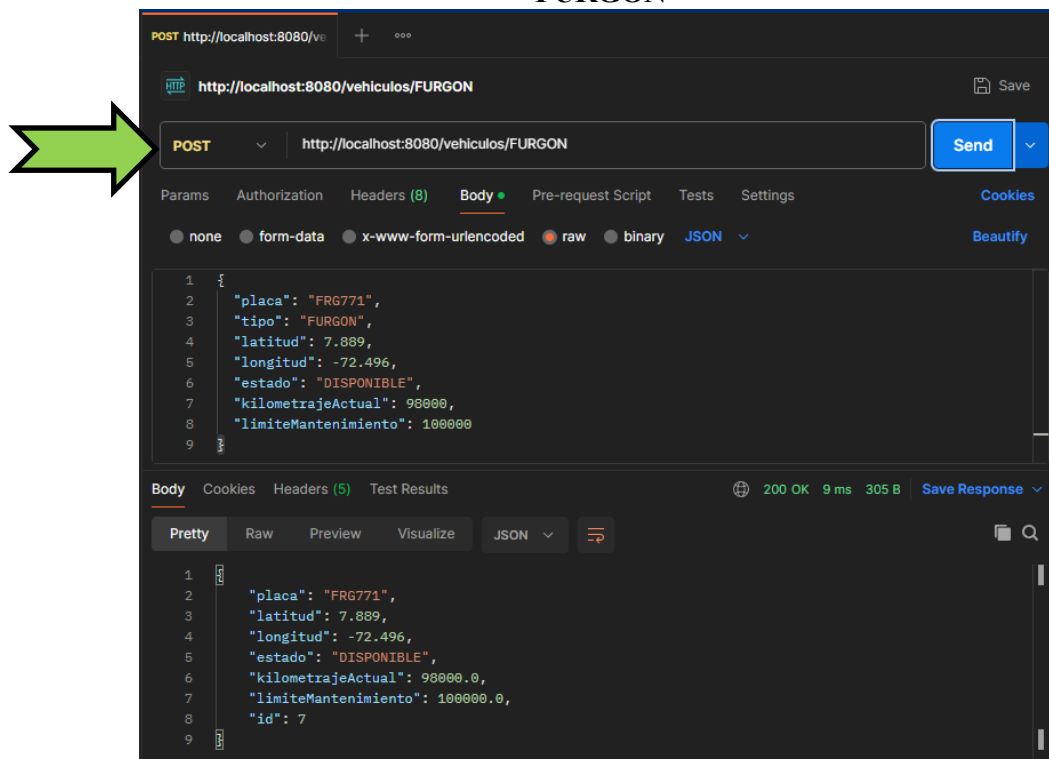
```
1 {
2   "placa": "MT0318",
3   "tipo": "MOTO",
4   "latitud": 4.609,
5   "longitud": -74.081,
6   "estado": "DISPONIBLE",
7   "kilometrajeActual": 8200,
8   "limiteMantenimiento": 10000
9 }
```

Body Cookies Headers (5) Test Results 200 OK 11 ms 303 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "placa": "MT0318",
3   "latitud": 4.609,
4   "longitud": -74.081,
5   "estado": "DISPONIBLE",
6   "kilometrajeActual": 8200.0,
7   "limiteMantenimiento": 10000.0,
8   "id": 6
9 }
```

FURGON



POST http://localhost:8080/vehiculos/FURGON

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

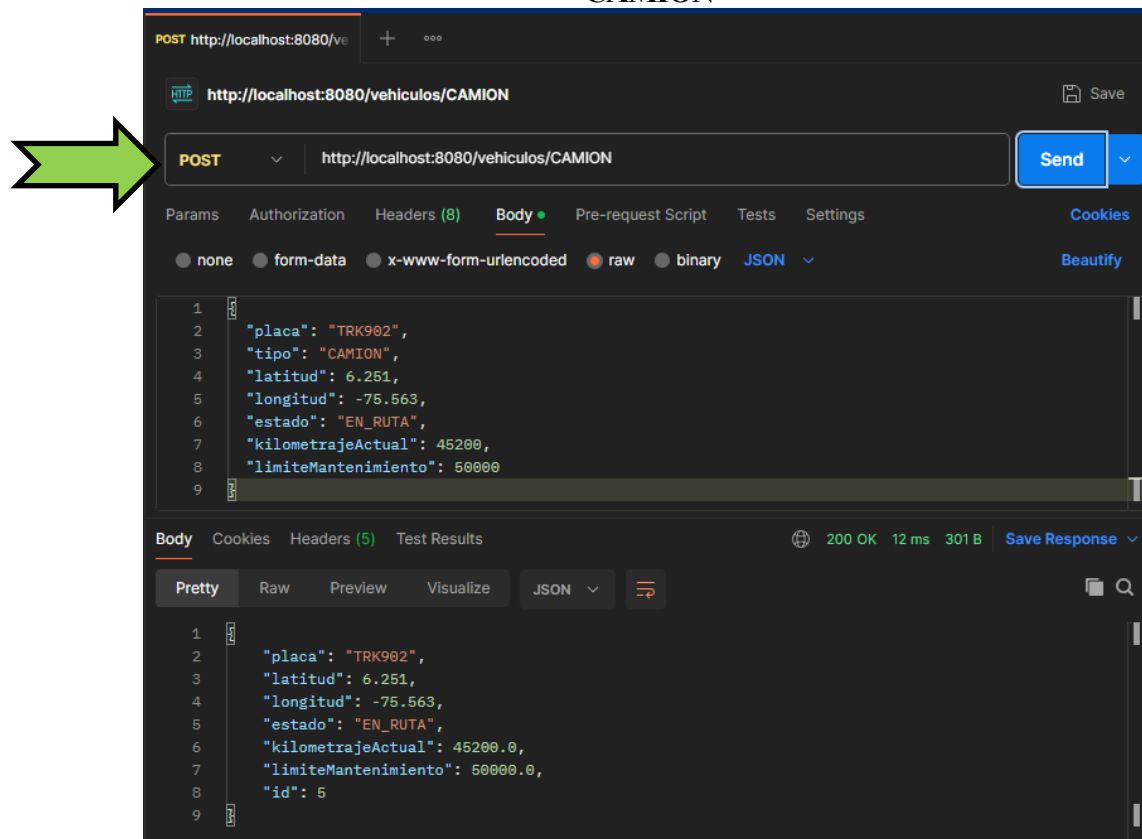
```
1 {
2   "placa": "FRG771",
3   "tipo": "FURGON",
4   "latitud": 7.889,
5   "longitud": -72.496,
6   "estado": "DISPONIBLE",
7   "kilometrajeActual": 98000,
8   "limiteMantenimiento": 100000
9 }
```

Body Cookies Headers (5) Test Results 200 OK 9 ms 305 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "placa": "FRG771",
3   "latitud": 7.889,
4   "longitud": -72.496,
5   "estado": "DISPONIBLE",
6   "kilometrajeActual": 98000.0,
7   "limiteMantenimiento": 100000.0,
8   "id": 7
9 }
```

CAMION



Implementamos el patrón Factory Method porque el sistema maneja múltiples tipos de vehículos con comportamientos específicos dentro del dominio logístico.

Actualmente el sistema contempla clases como:

- Camion
- Furgon
- Moto

Cada uno representa una especialización del concepto general `Vehiculo`, pero su proceso de creación puede variar según:

- Capacidad de carga
- Tipo de operación
- Configuración inicial
- Validaciones específicas

CONCLUSIONES

El Sistema de Gestión de Flotas incorpora de manera intencional y estructurada dos patrones de diseño fundamentales: Singleton y Factory Method, los cuales fortalecen la calidad arquitectónica, la mantenibilidad y la escalabilidad del sistema.

La implementación del patrón Singleton se evidencia en las clases de servicio gestionadas por el contenedor de Spring, particularmente aquellas anotadas con `@Service`. Este mecanismo garantiza que exista una única instancia por clase dentro del contexto de la aplicación, lo cual asegura coherencia en la ejecución de la lógica de negocio, optimización de recursos y control centralizado del comportamiento del sistema. En un entorno backend como el desarrollado, donde múltiples peticiones HTTP interactúan simultáneamente con la capa de servicios, el patrón Singleton permite mantener un estado consistente sin generar instancias innecesarias que incrementen el consumo de memoria o introduzcan inconsistencias operativas.

Por otra parte, el patrón Factory Method fue implementado para encapsular la creación de los distintos tipos de vehículos dentro del dominio. En lugar de instanciar directamente las clases concretas mediante operadores `new` dentro de los servicios, la responsabilidad de creación fue delegada a fábricas especializadas. Esta decisión elimina el acoplamiento directo entre el servicio y las clases concretas, permitiendo que el sistema sea extensible sin modificar su estructura central. Gracias a esta implementación, la incorporación de nuevos tipos de vehículos puede realizarse agregando nuevas clases y sus respectivas fábricas, sin alterar el código existente, reduciendo así el riesgo de introducir errores en funcionalidades ya probadas.

Ambos patrones trabajan de manera complementaria: el Singleton garantiza una gestión eficiente y controlada de los servicios, mientras que el Factory Method proporciona flexibilidad y extensibilidad en la construcción de objetos del dominio. En conjunto, su aplicación demuestra una comprensión adecuada de los principios de diseño orientado a objetos y una intención clara de construir un sistema desacoplado, escalable y alineado con buenas prácticas de ingeniería de software.

REFERENCIAS

Cockburn, A. (2005). *Hexagonal architecture*. Recuperado de <https://alistair.cockburn.us/hexagonal-architecture>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley. Versión en línea (Extracto disponible): https://en.wikipedia.org/wiki/Design_Patterns

Martin, R. C. (2003). *Agile software development: Principles, patterns, and practices*. Prentice Hall. Disponible en: <https://www.informit.com/articles/article.aspx?p=121615>

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall. Resumen y conceptos clave en: <https://8thlight.com/blog/uncle-bob/2012/08/13/the-clean-architecture.html>

IBM Developer. (s.f.). *Factory Method pattern*. Recuperado de <https://developer.ibm.com/articles/ws-factory/>

Microsoft Docs. (s.f.). *Design patterns – Singleton*. Recuperado de <https://learn.microsoft.com/en-us/dotnet/architecture/modern-design-patterns/singleton>